

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবিয়ার

(Prof. Chester Rebeiro)

কম্পিউটার সায়েন্স এন্ড ইঞ্জিনিয়ারিং

(Department of Computer Science and Engineering)

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি (Indian Institute of Technology), মাদ্রাজ

সপ্তাহ – 01 বক্তৃতা- 12

ক্রিয়েট, এক্সিকিউট এবং প্রসেস থেকে এগজিট

নমস্কার, এবং এই ভিডিওটিতে স্বাগত জানাই। এই ভিডিওতে আমরা দেখব কীভাবে ক্রিয়েট, এক্সিকিউট এবং প্রসেস থেকে এগজিট হয়।

(স্লাইডসময় পড়ুন: 00:27)

একটি প্রসেস তৈরি করার জন্য, অপারেটিং সিস্টেম ক্লোনিং নামে একটি কৌশল ব্যবহার করে। যখন একটি প্রসেস কাকতালীয় প্রসেস 1 এখানে একটি ফর্ক নামক সিস্টেম কল আহ্বান করে, এটি ফলে কার্নেল ড্রিগার্ড (triggered) হচ্ছে। কার্নেল তারপর কার্নেলের স্থানটিতে ফর্ক সিস্টেম কলটি এক্সিকিউট করে এবং একটি শিশু প্রসেস হিসাবে পরিচিত হয়; এখানে এটি প্রসেস 2 হিসাবে ধরে নেওয়া হচ্ছে। শিশু প্রসেসটি মূল প্রসেসের একটি সঠিক ডুপ্লিকেট বা নকল। সুতরাং, আসুন দেখি কীভাবে ফর্ক সিস্টেম কল একটি উদাহরণের সঙ্গে কাজ করে।

(স্লাইডটাইপ দেখুন: 01:04)

আসুন আমরা ধরে নিই যে আমরা এই বিশেষ প্রোগ্রামটি লিখেছি যা ফর্ক সিস্টেম কলকে আমন্ত্রণ জানিয়েছে। ফর্ক সিস্টেম কল থেকে প্রত্যাবর্তন পি এবং পি একটি পূর্ণসংখ্যা হিসাবে সংজ্ঞায়িত করা হয়, এবং এটির মান -1, 0 বা 1 এর চেয়ে বড় কিছু হতে পারে। সুতরাং, যখন এই নির্দিষ্ট প্রোগ্রাম এক্সিকিউট হয়, ফর্ক সিস্টেম কল অপারেটিং সিস্টেমকে এক্সিকিউট করার জন্য ড্রিগার করবে, এবং ওএস তারপর প্যারেন্ট প্রসেসের একটি সঠিক প্রতিরূপ তৈরি করবে। এবার এটা প্যারেন্ট প্রসেস হবে এবং এই প্রসেসটির সঠিক প্রতিরূপ শিশু প্রসেস হিসাবে পরিচিত হয়।

প্যারেন্ট প্রসেস এবং চাইল্ড প্রসেসের মধ্যে একমাত্র পার্থক্য হচ্ছে প্যারেন্ট প্রসেসে পি একটি মান যা চাইল্ড প্রসেসের PID হয়। সুতরাং, এই মান সাধারণত 0 এর চেয়ে বড়; যাইহোক, চাইল্ড প্রসেসে পি এর মান 0। সুতরাং, যখন ওএস তার কার্যকারিতা সমাপ্ত করে, প্যারেন্ট এবং সেইসাথে সদ্য গঠিত চাইল্ড উভয়ই তাদের বিভিন্ন মান পি কে সিস্টেম কল থেকে ফিরিয়ে দেবে। প্যারেন্ট প্রসেসে, চাইল্ডের PID 0 এর চেয়ে বড়, তাই প্যারেন্ট প্রসেস বিশেষ স্টেটমেন্টটি এক্সিকিউট করবে। যাইহোক, চাইল্ড প্রসেসে, পি এর মান 0 থেকে সমান হলে অন্যথায় এর অংশ এই সবুজ স্টেটমেন্টগুলি কার্যকর করা হবে। এখন যদি ফর্ক সিস্টেম কল ব্যর্থ হলে -1 এর একটি মান ফেরত হবে যা এই printf এর ফলে এই এররটি পর্দায় প্রদর্শিত হবে।

(স্লাইডটাইপ পড়ুন: 02:48)

এখন, দেখি কিভাবে এই ফর্ক সিস্টেম কল অপারেটিং সিস্টেমের ভিতরে কাজ করে। সুতরাং, আমরা যেমন আগে উল্লেখ করেছি, প্রতিটি প্রসেসের কার্নেলে তিনটি মেটাডেটা সংরক্ষিত আছে। সুতরাং, একটি পেজ টেবিল, দ্বিতীয়টি কার্নেল স্ট্যাক, এবং তৃতীয়টি proc স্ট্রাকচারে প্রসেস নিয়ন্ত্রণ ব্লক - PCB- এর মধ্যে একটি এন্ট্রি। সুতরাং, যখন ফর্ক সিস্টেম কল এক্সিকিউট হয়, পেজ টেবিলের একটি কপি তৈরি করা হয়। তাই, এটি তৈরি করা হচ্ছে এমন নতুন চাইল্ডের পেজ টেবিলের অনুরূপ। একইভাবে, প্যারেন্ট প্রসেসটির কার্নেল স্ট্যাকটি চাইল্ডটির কার্নেল স্ট্যাকের মত ডুপ্লিকেট করা হয়। আরও একটি নতুন পিসিবি এন্ট্রি চাইল্ডের পিসিবি এন্ট্রির অনুরূপে তৈরি করা হয়। এটি করার জন্য, অপারেটিং সিস্টেম দ্বারা প্রথমে কোনও অব্যবহৃত PID খুঁজে পাওয়া প্রয়োজন। তারপর সদ্য গঠিত প্রসেস বা চাইল্ড প্রসেসটির স্টেট নিউ তে সেট করা হয়। তারপর PCB- তে অনেকগুলি পয়েন্টার থাকে যেমন pagetable পয়েন্টার অর্থাৎ পেজ টেবিলের পয়েন্টারটি এই নির্দিষ্ট পেজ টেবিলের দিকে নির্দেশ করবে, কার্নেল স্ট্যাক পয়েন্টার এই এক নতুন গঠিত অনুলিপি এন্ট্রিকে নির্দেশ করে, এবং trapframe এর একটি পয়েন্টার এবং একটি কন্ট্রোল যা এই কার্নেল স্ট্যাকের মধ্যে উপস্থিত। অন্যান্য তথ্য যেমন ফাইল থোলা, প্রসেসের আকার, বর্তমান কার্যকরী ডিরেক্টরি ইত্যাদি প্যারেন্ট এর পিসিবি থেকেও চাইল্ড এর পিসিবিতে কপি করা হয়।

(স্লাইডটাইপ পড়ুন: 04:32)

অবশেষে ফিরে আসার আগে, অপারেটিং সিস্টেমটি নতুন তৈরি চাইল্ডটির স্টেট READY তে সেট করবে। সুতরাং, আমরা দেখতে পাচ্ছি যে ফর্কটির এক্সিকিউশনের প্রারম্ভে, চাইল্ডের স্টেটটি NEW তে সেট করা হয়েছে যখন ফর্কের এক্সিকিউশন শেষ, ওএসটি স্টেটটি READY টিই সেট করবে। সুতরাং, কেন আমাদের এই মধ্যবর্তী স্টেট NEW প্রয়োজন? সুতরাং, NEW কি বোঝায় এবং READY কি বোঝায়। তাই, NEW স্টেটটি ইঙ্গিত করে যে পিআইডি, এখানে নির্বাচিত পিআইডি ইতিমধ্যেই নেওয়া হয়েছে এবং প্রসেসটি বর্তমানে তৈরি করা হচ্ছে কিন্তু চালানোর জন্য প্রস্তুত নয়। অন্য দিকে, যখন স্টেটটি READY তখন এটির মানে হল যে কার্নেলে বিভিন্ন

মেটাডেটার সূচনা করা হয়েছে, এবং প্রসেসটিকে রেডি কিউতে স্থানান্তরিত করা যেতে পারে এবং এটি এক্সিকিউট করার জন্য প্রস্তুত।

(স্লাইডটাইপ পড়ুন: 05:31)

সুতরাং, রেডি কিউতে নতুন প্রসেসের একটি এন্ট্রি থাকবে এবং যখনই সিপিইউ সময়সূচী নির্ধারিত হয় সিডিউলার তখন এটি এই নির্দিষ্ট প্রসেসটি বেছে নিয়ে ড্রিগার করে এবং তার অবস্থাটি READY থেকে RUNNING এ পরিবর্তন করতে পারে এক্সিকিউট করার জন্য, এবং নতুন প্রসেস প্রসেসর মধ্যে এক্সিকিউট করতে শুরু করবে।

(স্লাইডটাইপ দেখুন: 05:57)

এখন কোনও প্যারেন্ট প্রসেস এবং চাইল্ড প্রসেসের ফর্কের মধ্যে একটি গুরুত্বপূর্ণ পার্থক্য হল, প্যারেন্ট প্রসেস ফর্কের মধ্যে চাইল্ডটির পিআইডি ফিরে আসে; যখন চাইল্ড প্রসেসের মধ্যে, ফর্ক সিস্টেম কল 0 ফিরিয়ে দেবে। সুতরাং, কিভাবে অপারেটিং সিস্টেম এই পার্থক্য করে? সুতরাং, মূলত ফর্ক সিস্টেম কল থেকে ফিরে কার্নেল স্ট্যাকের অংশ হিসেবে সংরক্ষণ করা হয়। সুতরাং, যখন ফর্ক এক্সিকিউট হয়, তখন ওএস বা অপারেটিং সিস্টেমটি প্যারেন্ট প্রসেসের কার্নেল স্ট্যাকের রিটার্ন মান সেট করবে চাইল্ডটির PID হিসাবে। উপরন্তু চাইল্ডের প্রসেসের কার্নেল স্ট্যাকে রিটার্ন মান 0 সেট করা হয়; এইভাবে যখন এই ক্ষেত্রে প্রতিটিতে সিস্টেম কল ফেরত আসে, প্রতিটি প্রসেস প্যারেন্ট প্রসেসের পাশাপাশি চাইল্ড প্রসেসের রিটার্ন টাইপের বিভিন্ন মান পাবে।

(স্লাইডটাইপ দেখুন: 06:59)

এখন, ফর্ককে শুরু করার সময় প্রাথমিক দিকগুলির মধ্যে একটি হল পেজ টেবিলের প্রতিলিপি। সুতরাং, আমাদের এখানে প্যারেন্ট পেজ টেবিল আছে, এবং যখন ফর্কটিকে ডাকা হয় তখন এই পেজ টেবিলের একটি ডুপ্লিকেট চাইল্ডটির জন্য তৈরি করা হয়। সুতরাং, এই ডুপ্লিকেট পেজ টেবিলের অর্থ কী? সুতরাং, যখন আমরা আসলে এই বিশেষ চিত্রটি দেখি, আমরা দেখতে পাই যে আমাদের দুটি পেজ টেবিলের একে অপরের প্রতিক্রিয়া এবং ঠিক একই এন্ট্রি আছে। সুতরাং, এর মানে কি এই যে, প্যারেন্ট এবং চাইল্ড পেজ টেবিল উভয়ই একই পেজ ফ্রেমের দিকে ইঙ্গিত করছে। তাই আসুন আরো বিস্তারিতভাবে এটা দেখি।

(স্লাইডটাইপ পড়ুন: 07:46)

তাই আমাদের প্যারেন্ট প্রসেসটি তার ভার্চুয়াল মেমোরির ম্যাপ এবং তার নিজস্ব ভার্চুয়াল মেমোরি ম্যাপের সাথে সংশ্লিষ্ট চাইল্ড প্রসেস; এবং এই প্রতিটি প্রসেসর নিজস্ব পেজ টেবিল আছে। সুতরাং, চাইল্ড প্রসেস পেজ টেবিলটি প্যারেন্ট প্রসেসের পেজ টেবিলের একটি সঠিক প্রতিরূপ। সুতরাং, এর মানে হল যে, প্যারেন্টের ব্লক 1 আর চাইল্ডের ব্লক 1 টি একই পেজ ফ্রেমকে পয়েন্ট করে যেটা হল 6। একইভাবে, প্যারেন্টের ব্লক 2 আর চাইল্ডের ব্লক 2 একই পেজ ফ্রেম 13 কে পয়েন্ট করে যেমন এখানে দেখানো হয়েছে এরূপে চাইল্ড প্রসেসের পেজ টেবিলটি এখানে দেখানো হয়েছে। সুতরাং, মূলত আমরা এখানে এই অর্জন করলাম যে আমাদের দুটি ভার্চুয়াল মেমরি ম্যাপ আছে, একটি প্রসেসের জন্য এক এবং আরেকটি চাইল্ডের জন্য এক। যাইহোক, RAM-র মধ্যে, আমাদের কাছে শুধুমাত্র প্যারেন্ট প্রসেসের এক কপি কোড এবং সংশ্লিষ্ট ডেটা আছে, প্যারেন্ট ও চাইল্ড পেজের টেবিল উভয়ই র‍্যামের একই পেজ ফ্রেমকে নির্দেশ করে।

(স্লাইডসময় পড়ুন: 08:58)

সুতরাং আসুন একটি উদাহরণ দেখা যাক। ধরুন, আমরা এই নিম্নোক্ত প্রোগ্রামটি লিখেছি যা ফর্ককে ডেকেছে এবং ফর্ক একটি pid ফেরত দিয়েছে; এবং প্যারেন্ট প্রসেসের মধ্যে, আমরা কিছু সময়ের জন্য ঘুমাই এবং তারপর printf আহ্বান করি যা i এর মান প্রিন্ট করে। সুতরাং, 'i' এখানে একটি পূর্ণসংখ্যা যার মান 23। একইভাবে, চাইল্ড প্রসেস যা এই else অংশে এক্সিকিউট করা হয়, আমরা আবার i এর মান প্রিন্ট করি। এখন যেহেতু চাইল্ডটি প্যারেন্টের একটি সঠিক প্রতিরূপ তাই i এর মান প্যারেন্টের সাথে সাথে চাইল্ডেও একই হবে। এইভাবে যখন আমরা এই প্রোগ্রাম এক্সিকিউট করি, আমরা এরকম কিছু পেতে পারি; চাইল্ড ও প্যারেন্ট উভয়ই i এর মান 23 প্রিন্ট করে।

(স্লাইডসময় পড়ুন: 09:49)

এখন, এই উদাহরণটি একটু পরিবর্তন করি এবং দেখি যদি আমরা এই নির্দিষ্ট লাইনটি যোগ করি তবে কি হবে। আমরা এখানে কি করেছি তা হল যে আমরা বাড়িয়েছি i, i ইকুয়াল টু i প্লাস 1 শুধুমাত্র চাইল্ড প্রসেসের মধ্যে। মনে রাখবেন যে এখানে প্যারেন্ট প্রসেসে, একটি ঘুম 1 আছে, যার মানে হল যে চাইল্ড প্রসেসে আমরা এই i ইকুয়াল টু i প্লাস 1 এর এক্সিকিউটের জন্য যথার্থ সমান সময় প্রদান করছি তারপর প্যারেন্টের printf টি এক্সিকিউট হবে। সুতরাং, আপনি কি মনে করেন এই বিশেষ প্রোগ্রাম আউটপুট কি হবে? কেউ আশা করতে পারে যে চাইল্ডটি 24 প্রিন্ট করবে, যেহেতু আমরা i এর মান i প্লাস 1 দ্বারা বাড়িয়েছি তাই 23 প্লাস 1; এবং প্যারেন্টও 24 এর প্রিন্ট করবে। যাইহোক, যখন আমরা এই প্রোগ্রামটি এক্সিকিউট করব তখন আমরা চাইল্ডের আপডেটেড মান 24 পেতে পারি; যাইহোক, প্যারেন্টে এখনও পুরানো মান 23 আছে। এখন দেওয়া আছে যে প্যারেন্ট এবং চাইল্ড উভয়ই

RAM- র মধ্যে একই পেজ ফ্রেমকে পয়েন্ট করে। তাই কিভাবে এটি সম্ভব? সুতরাং, আমাদের কেন এই ধরনের ঘটনা ঘটেছে তা দেখতে হবে।

(স্লাইডটাইপ পড়ুন: 11:04)

সুতরাং, এই ঘটনা অপারেটিং সিস্টেমের মধ্যে প্রয়োগ করা হয় কপি অন রাইট (COW) হিসেবে পরিচিত একটি প্রযুক্তির কারণে ঘটে। সুতরাং, যখন একটি ফর্ক সিস্টেম কল OS এ এক্সিকিউট হয়, সমস্ত প্যারেন্ট পেজগুলি প্রাথমিকভাবে শেয়ার হিসাবে চিহ্নিত করা হয়। শেয়ার করা বিটটি প্যারেন্ট এবং চাইল্ডের পেজ টেবিলের মধ্যে উপস্থিত। যখন শেয়ার করা কোনও পেজের ডেটা পরিবর্তন হয়, তখন OS টি বিরতি দেয় এবং সেই পেজটির অনুলিপি তৈরি করে। এইভাবে প্যারেন্ট এবং চাইল্ডের সেই পেজগুলির বিভিন্ন কপি থাকবে এবং মনে রাখবেন যে 'এই' পেজটি এখানে তুলে ধরা হয়েছে, তাই; এর অর্থ, কেবলমাত্র সেই পেজটি প্যারেন্ট এবং চাইল্ডের মধ্যে ভিন্ন হবে যখন অন্য সব পেজগুলি একই থাকবে। আসুন দেখি আমাদের উদাহরণের সাথে কিভাবে কপি অন রাইট কাজ করে। আসুন আমরা বলি যে আমি i এর মানটি একটি নির্দিষ্ট পেজ ফ্রেমের মধ্যে সংরক্ষিত আছে, যা প্যারেন্ট পেজ টেবিলের পাশাপাশি চাইল্ড পেজ টেবিলের দ্বারা নির্দেশিত। যখন চাইল্ড প্রসেসটি এক্সিকিউট হয় i এর মান i প্লাস 1 এ ইনক্রিমেন্ট হয় আর এর মান 23 থেকে বেড়ে 24 হয় এবং নতুন মান ফিরে আসে, তখন ওএসটি লিখতে বাধা দেবে এবং চাইল্ডটির জন্য একটি নতুন পেজ তৈরি করবে।

(স্লাইডটাইপ পড়ুন: 12:২5)

তাই, চাইল্ড প্রসেসের i এখানে উপস্থিত থাকবে এবং এটির নতুন আপডেট মান পাবে 24 যেখানে প্যারেন্টের i এর মূল মান এখানে উপস্থিত হবে এবং পুরানো i এর মান 23 থাকবে। আরও, চাইল্ডের পেজ টেবিলের সংশ্লিষ্ট পেজ এন্ট্রি তারপর আপডেট করা হবে। সুতরাং, COW এর সুবিধা কি, আমরা পরবর্তী স্লাইডে দেখব। এখন আমরা একটি প্রোগ্রাম ক্লোন কিভাবে করে তে দেখলাম, আমরা এখন দেখবো একটি সম্পূর্ণ নতুন প্রোগ্রাম কিভাবে এক্সিকিউট হবে।

(স্লাইডসময় পড়ুন: 13:05)

একটি নতুন প্রোগ্রাম এক্সিকিউশন দুটি ধাপ দ্বারা গঠিত; প্রথমে একটি ফর্ক সিস্টেম কল চালু করা প্রয়োজন যার ফলে একটি চাইল্ডের প্রসেস তৈরি হবে যা প্যারেন্ট প্রসেসের একটি সঠিক প্রতিক্রিয়া, এবং তারপর একটি exec সিস্টেম কল চালু করা প্রয়োজন যার ফলে নতুন প্রোগ্রামটি এক্সিকিউট হবে। সুতরাং, আসুন আমরা এই বিশেষ ছোট সি কোড টি বিবেচনা করি। সুতরাং, আমরা কি দেখতে পাচ্ছি যে প্রাথমিকভাবে একটি ফর্ক কল করা হয় যা একটি PID মান ফিরিয়ে দেয় এবং প্যারেন্ট প্রসেসের অনুলিপি তৈরি করবে যার নাম চাইল্ড প্রসেস। এখন চাইল্ড প্রসেসের মধ্যে, PID এর মান 0 থাকে, সুতরাং এক্সিকিউশন এই else অংশটিতে প্রবেশ করবে। এখন

চাইল্ডের প্রসেসের মধ্যে, আমরা এই ক্ষেত্রে একটি সিস্টেম কল `exec` আহ্বান করি এটি `execlp` হিসাবে পরিচিত এবং বেশ কয়েকটি আর্গুমেন্ট পাস করে, এর সবচেয়ে গুরুত্বপূর্ণ বৈশিষ্ট্য হল অবশ্যই এক্সিকিউটেবল ফাইলের নাম। আমাদের ক্ষেত্রে, আমরা একটি ডট আউট ফাইল চালাবো চেষ্টা করছি। সুতরাং, যখন এই `exec` সিস্টেম কল এক্সিকিউট হয় এটি অপারেটিং সিস্টেম কার্যকারিতাকে ট্রিগার করে। তাই, অপারেটিং সিস্টেম হার্ডডিস্কে একটি ডট আউটের সঠিক অবস্থান খুঁজে পায় এবং এটি চাহিদার উপর নির্ভর করে একটি ডট আউট এক্সিকিউটের জন্য প্রয়োজনীয় পেজগুলিকে লোড করে।

(স্লাইডসময় পড়ুন: 14:28)

তাই আসুন আমরা এই আরো সচিত্র ব্যাপারটা দেখে। সুতরাং, যখন আমরা ফর্ক দ্বারা ডাকি তখন কী হয়? সুতরাং, আমরা জানি যে ফর্ক একটি চাইল্ড প্রসেস তৈরি করবে, যা প্যারেন্টের একটি সঠিক প্রতিরূপ। তাই, আমরা যা দেখেছি তা হল চাইল্ডটির নিজস্ব পেজ টেবিল রয়েছে, কিন্তু পেজ টেবিলের সমস্ত এন্ট্রিগুলি একই পেজ ফ্রেমের প্যারেন্ট হিসাবে ম্যাপ করে। এখন আসুন আমরা দেখি কি কি ঘটে যখন `exec` সিস্টেম কল আসে। সুতরাং, যখন `exec` সিস্টেম কল আহ্বান করা হয়, অপারেটিং সিস্টেম হার্ড ডিস্কের মধ্যে প্রোগ্রাম বা এক্সিকিউটেবল অবস্থান খোজে এবং চাহিদার উপর নির্ভর করে হার্ডডিস্কের ব্লকগুলি পেজ ফ্রেমের মধ্যে লোড করে। উদাহরণস্বরূপ, যখন আমরা নতুন চাইল্ড প্রসেস এক্সিকিউশন শুরু করি, তখন এটি নতুন চাইল্ডের প্রথম ব্লকটি লোড করবে; এবং তারপর চাহিদা মতো যখন প্রয়োজন নতুন ব্লক RAM এ লোড করা হবে। পরবর্তীকালে, চাইল্ডের প্রসেসের পেজ টেবিলটিও প্রয়োজন অনুসারে আপডেট করা হবে। তাই, আমরা দুটি জিনিস দেখি যা ঘটে, প্রথমে আমরা লক্ষ্য করি যে যখনই কোনও নতুন ব্লক বা যখনই চাইল্ড প্রোগ্রামটি এক্সিকিউট করা হচ্ছে তখন একটি নতুন কার্যকারিতা থাকে যা একটি নতুন পেজ ফ্রেম চাইল্ড প্রসেসে বরাদ্দ করে দেয়। যাইহোক, যা কার্যকারিতা যা প্যারেন্ট এবং চাইল্ড উভয়েরই আছে, এখনও সাধারণ পেজগুলি ভাগ করে নেয়, যেমনটি এইরকম উদাহরণস্বরূপ।

(স্লাইডটাইপ পড়ুন: 15:54)

তাই আসুন দেখি কপি অন রাইট এর সুবিধা কি? সুতরাং, আমরা জানি যে বেশিরভাগ প্রোগ্রামগুলি বেশ কয়েকটি কার্যকারিতা সহ লিখিত আছে যা স্বাভাবিক। উদাহরণস্বরূপ, আমরা জানি যে অনেকগুলি প্রোগ্রাম `printf` বা `scanf` বা স্ট্যান্ডার্ড লাইব্রেরী(standard library)ফাংশন যেমন `freadf`, `fscanf` এবং `fprintf` ব্যবহার করবে। যেহেতু, একটি নতুন প্রসেসকে প্যারেন্টের প্রথম ক্লোনিং করে তৈরী করা হয় এবং তারপর নতুন প্রোগ্রামটিকে এক্সিকিউট করা হয়, এইভাবে প্যারেন্টটির কার্যকারিতা অনেকগুলি চাইল্ড প্রসেসেও উপস্থিত হবে। এখন যেহেতু পেজগুলি শুধুমাত্র চাইল্ড প্রসেসের চাহিদার উপর প্রতিস্থাপিত হয় তাই সাধারণ কার্যকারিতা বা সাধারণ কোড যা প্যারেন্ট এবং চাইল্ড প্রসেস উভয়েই বিদ্যমান, এখনও দুটি প্রসেসের মধ্যে ভাগ করা আছে।

উদাহরণস্বরূপ, printf যা প্যারেন্ট প্রসেসের মধ্যে উপস্থিত থাকে এবং printf যা চাইল্ড প্রসেসে উপস্থিত থাকে, তখনও ঠিক RAM এর একই পেজ ফ্রেমে বা ফিজিক্যাল মেমরিতে নির্দেশ করে। তাই, এর মানে হল যে যদিও আপনার সিস্টেমে 100 টি ভিন্ন প্রসেস চলছে, এবং এই সমস্ত প্রসেসগুলি একটি সাধারণ ফাংশন ব্যবহার করতে পারে যেমন printf, RAM-তে যেমন printf এর একটিমাত্র কপি থাকবে। তারপর সমস্ত প্রসেসগুলি তাদের পেজ টেবিলের এই নির্দিষ্ট পেজ ফ্রেমকে নির্দেশ করবে, যাতে printf রয়েছে।

(স্লাইডটাইপ দেখুন: 17:২1)

তাই আমরা দেখেছি যে একটি নতুন প্রসেস তৈরি করার জন্য প্রথমে একটি ফর্ক দরকার এবং যা একটি exec সিস্টেম কল দ্বারা অনুসরণ করা হয়। সুতরাং, সিস্টেমের প্রতিটি প্রসেস এইভাবে নির্মিত হয়। অতএব আমরা একটি বৃক্ষের মত কাঠামো পাই যেখানে আপনার একটি মূল নোড আছে যা init ডট ডি নামে পরিচিত এবং প্রতিটি পরবর্তী নড সিস্টেমের চলমান প্রসেসের প্রতিনিধিত্ব করে এবং পূর্বের প্রসেস থেকে তৈরি হয়। উদাহরণস্বরূপ, প্রসেস compiz হল চাইল্ড প্রসেসের জীনোম-সেশন। পরিবর্তে gnome- সেশন একটি চাইল্ড init এর প্রসেস। সুতরাং, অবশেষে আমরা রুট প্রসেসটিতে পৌঁছই যা এই ক্ষেত্রে inti ডট ড হিসাবে পরিচিত। যদি আপনি আগ্রহী থাকেন, তবে আপনি এই বিশেষ কমান্ডটি দেখতে পারেন বা এক্সিকিউসন করতে পারেন যা আপনার bash প্রম্পট থেকে pstree যা আপনার সিস্টেমের সমস্ত প্রসেসগুলি একটি বৃক্ষের মতো কাঠামোর মধ্যে তালিকা করে। সুতরাং, আমরা দেখছি যে সিস্টেমের প্রতিটি প্রসেসকে এক্সিকিউট করার জন্য একটি প্যারেন্ট প্রসেস আছে। সুতরাং, প্রথম প্রসেসটি কি? এটি একটি সিস্টেমের একমাত্র প্রসেস, যার কোনও প্যারেন্ট নেই। সুতরাং, কে এই প্রসেস তৈরি করে?

(স্লাইডটাইপ পড়ুন: 18:33)

xv6-র প্রথম প্রসেস, প্রথম প্রসেসটি initcode dot s- এ উপস্থিত থাকে, অন্যান্য প্রসেসের মত নয়, কারণ বুটিং প্রসেসের সময় কার্নেল দ্বারা এই নির্দিষ্ট প্রসেসটি তৈরি করা হয়। সুতরাং, ইউনিক্স অপারেটিং সিস্টেমে, প্রথম প্রসেসটি স্ল্যাশ sbin স্ল্যাশ init- এ উপস্থিত থাকে। সুতরাং, যখন আপনি আপনার সিস্টেম চালু করেন এবং অপারেটিং সিস্টেমটি বুট করতে শুরু করেন, এটি আপনার সিস্টেমের বিভিন্ন উপাদানগুলি শুরু করে এবং অবশেষে এটি প্রথম ব্যবহারকারী প্রসেসটি তৈরি করে যা কার্যকর হয়। সুতরাং, এটি কখনও কখনও একটি সুপার প্যারেন্ট হিসাবে পরিচিত হয়, এবং এর প্রধান কাজ আসলে অন্য প্রসেসকে ফর্ক করা। সাধারণত লিনাক্সে এই প্রথম প্রসেসটি স্ল্যাশ etc স্ল্যাশ init dot d তে উপস্থিত কিছু স্ক্রিপ্ট শুরু করবে।

(স্লাইডটাইপ দেখুন: 19:২9)

এখন আসুন কোড `init dot c` দেখুন, যা `xv6` এর অংশ। মূলত এই বিশেষ কোড একটি প্রসেসকে ফর্ক করে এবং একটি শেল `sh` তৈরি করে। সুতরাং, এটি কোডের টুকিটাকি, বিশেষ করে আমরা চাই আপনি এই বিশেষ লুপটি লক্ষ্য করুন, এই বিশেষ `for` লুপটি, যা একটি অসীম `for` লুপ। সুতরাং, এটি অসীম পর্যন্ত চলে; এটির একমাত্র কাজ হচ্ছে একটি চাইল্ড প্রসেস তৈরি করার জন্য একটি প্রসেসকে ফর্ক করা; এবং চাইল্ড প্রসেসে, এটি `exec sh` চালায় যা কিছু আরগুমেন্ট সহ একটি শেল। তারপর যে নির্দিষ্ট ফর্কড প্রসেস সম্পন্ন পর্যন্ত এটি অপেক্ষা করে, এবং তারপর এই জিনিস চিরতরে চলতে থাকে।

(স্লাইডটাইম পড়ুন: 20:16)

এখন আমরা দেখলাম কিভাবে প্রসেসগুলি ক্লোন করা হয়, কিভাবে একটি নতুন প্রোগ্রাম এক্সিকিউট করা হয়। আসুন দেখি কিভাবে একটি প্রসেস শেষ করা যায়।

(স্লাইডটাইপ পড়ুন: 20:27)

একটি প্রসেস একটি `exit` কল হিসাবে পরিচিত কিছু দ্বারা সমাপ্ত করা হয়। তাই, উদাহরণস্বরূপ, আসুন আমরা আমাদের উদাহরণে ফিরে যাই। এবং চাইল্ড প্রসেসের মধ্যে, আমরা আসলে চালাই এক্সিকিউটেবল এ ডট আউট বা এক্সিকিউট, যা তারপর এক্সিকিউট হয় এবং এই একটি `exit(0)` দ্বারা অনুসরণ করা হয়। সুতরাং, এই `exit` চাইল্ড প্রসেসে প্রয়োগ করা হয় এবং প্যারামিটার 0 একটি মর্যাদা যা প্যারেন্ট প্রসেসে প্রেরিত হয়। সুতরাং, একটি প্রসেস সমাপ্ত করার এই বিশেষ উপায় একটি স্বেচ্ছাসেবী পরিসমাপ্তি হিসাবে পরিচিত হয়।

(স্লাইডটাইপ পড়ুন: 21:03)

স্বেচ্ছাসেবী অবসান ছাড়াও, একটি অযৌক্তিক অবসান হিসাবেও পরিচিত কিছু আছে। যেমন একটি পরিসমাপ্তিতে, প্রসেসকে জোরপূর্বক সমাপ্ত করা হয়। সুতরাং, একটি সিস্টেম কল যা প্রকৃতপক্ষে এটি `kill` সিস্টেম কল, যা দুটি প্যারামিটার `pid` লাগে যা প্রসেস `pid` এবং যাকে `kill` করা প্রয়োজন এবং একটি সংকেত বা সিগন্যাল। একটি সংকেত হল একটি অসিঙ্ক্রোনাস বার্তা যা অপারেটিং সিস্টেম থেকে প্রেরিত হয় বা সিস্টেমের মধ্যে চলা অন্য কোনও প্রসেস। বিভিন্ন ধরনের সিগন্যাল রয়েছে যেমন `SIGTERM`, `SIGQUIT`, `SIGINT` এবং `SIGHUP`। যখন একটি প্রসেস `SIGQUIT` মত একটি সংকেত পায়, প্রসেসটি অপ্রাসঙ্গিক ভাবে হলেও শেষ করা হয়।



(স্লাইডটাইপ পড়ুন: 21:48)

সুতরাং, প্রসেসটি বন্ধ করার বিষয়ে আরেকটি গুরুত্বপূর্ণ সিস্টেম কল হল wait সিস্টেম কল। wait সিস্টেম কল প্যারেন্ট দ্বারা আহ্বান করা হয়; এটি তার সন্তানদের মধ্যে একজন exit না হওয়া পর্যন্ত প্যারেন্ট একটি ব্লকড (blocked) অবস্থায় থাকে। যদি প্যারেন্ট এর কোন চলন্ত চাইল্ড না থাকে তবে -1 ফিরে আসবে। সুতরাং, এবার আমাদের উদাহরণ ফিরে যান, যেখানে প্যারেন্ট একটি চাইল্ডের প্রসেসকে ফর্ক করেছে; এবং প্যারেন্ট প্রসেসে, এটি একটি PID সংগ্রহ করে যা চাইল্ড প্রসেসের PID এর সমান। সুতরাং, এর ফলে প্যারেন্ট প্রকৃতপক্ষে wait এ থাকা স্টেটমেন্টগুলোকে এক্সিকিউট করবে; এর ফলে প্যারেন্ট প্রসেসটি ব্লক হবে যতক্ষণ না পর্যন্ত চাইল্ড প্রসেসটি প্রস্থান করছে। যখন চাইল্ড প্রসেসটি প্রস্থান করে, তখন এটি প্যারেন্ট প্রসেসটিকে জাগিয়ে তুলবে এবং wait ফাংশনটি পিআইডি এর রিটার্ন মান দিয়ে ফেরত হবে, যা চাইল্ড প্রসেসের pid। সুতরাং, এই বিশেষ উদাহরণে চাইল্ডের রিটার্ন স্টেটাস 0 প্রাপ্ত করার জন্য, প্যারেন্ট প্রসেস এই যেমন একটি wait আহ্বান করতে পারেন। সুতরাং, এই বিশেষ রূপে, একটি পয়েন্টার অবস্থাতে প্রেরণ করা হয় যেখানে অপারেটিং সিস্টেম চাইল্ড প্রসেসে exit স্টেটাস স্থাপন করবে।

(স্লাইডসময় পড়ুন: 23:15)

যখন একটি প্রসেস শেষ হয়, এটি একটি জম্বি(Zombie) বা একটি পরিত্যক্ত প্রসেস হিসাবে পরিচিত হয়। একটি zombie প্রসেসের বিশেষত্ব হল যে নির্দিষ্ট প্রসেস আর এক্সিকিউট করা হয় না; যাইহোক, অপারেটিং সিস্টেমের মধ্যে প্রসেস নিয়ন্ত্রণ ব্লক এখনও অব্যাহত থাকবে। সুতরাং, কেন আমাদের অপারেটিং সিস্টেমের মধ্যে zombies এর এই ধারণা আছে? সুতরাং, Zombie উপস্থিত রয়েছে, যাতে প্যারেন্ট প্রসেস wait সিস্টেম কল এর মাধ্যমে চাইল্ড এর exit অবস্থা পড়তে পারে। যখন একটি নির্দিষ্ট প্রোগ্রাম প্রস্থান করে, তখন তার exit অবস্থাটি অপারেটিং সিস্টেমের পিসিবি এর মধ্যে উপস্থিত থাকে। সুতরাং, প্যারেন্ট প্রসেস দ্বারা wait সিস্টেম কল চালু করা হলে, বর্তমান চাইল্ড প্রসেসটির পিসিবি পড়বে এবং এর exit স্ট্যাটাসটি সেখানে থেকে নেওয়া হবে। যখন wait সিস্টেমে কলটি মূলত প্যারেন্ট দ্বারা ডাকা হয়, এবং OS- এ উপস্থিত অতিরিক্ত Zombie এন্ট্রিগুলি সরানো হবে। এটি প্রসেস রিপড(reaped) হিসাবে পরিচিত হয়। তাই, প্যারেন্ট চাইল্ডের অবস্থা সম্পর্কে না পড়ে তাহলে কি হবে? এই ক্ষেত্রে, এটি একটি রিসোর্স লিকেজ(leakage) হবে এবং অপারেটিং সিস্টেমের Zombie এন্ট্রিগুলি অসীমভাবে বিদ্যমান থাকবে। সুতরাং, এইগুলি অবশেষে রিপার প্রসেস নামে পরিচিত অপারেটিং সিস্টেমের একটি বিশেষ প্রসেসের দ্বারা মুছে ফেলা হয়। অপারেটিং রিপার প্রসেসটি পর্যায়ক্রমিকভাবে চালায় এবং OS- এ উপস্থিত যে কোনও জম্বি প্রসেসের অবস্থার পুনরুদ্ধার করে।

(স্লাইডসময় পড়ুন: 24:45)

যখন একটি প্যারেন্ট প্রসেস তার চাইল্ডের আগে বন্ধ হয়, এর ফলে এটি একটি অনাথ বা অরফান (Orphan) হিসাবে পরিচিত হয়। উদাহরণস্বরূপ, এটিকে প্রসেস ট্রী হিসাবে বিবেচনা করুন এবং বিশেষ করে এটি একটি প্যারেন্টের সাথে একটি প্রসেস যা এখানে আছে। এখন এই বিশেষ প্যারেন্ট বর্তমান যতক্ষণ চাইল্ডটি এক্সিকিউট করতে থাকে, তখন শিশুটি একটি অনাথ হিসাবে পরিচিত হয়। এই ক্ষেত্রে, sbin init প্রসেসের প্রথম প্রসেসটি অনাথ চাইল্ডটিকে গ্রহণ করবে।

(স্লাইডটাইপ পড়ুন: 25:20)

দুই ধরনের অনাথ আছে; এক যখন প্যারেন্ট ক্র্যাশ ঘটে যখন আনইনটেনশন্যাল (Unintentional) বা অনিচ্ছাকৃত অনাথ হয়। অন্যটি হল ইন্টেনশনাল বা ইচ্ছাকৃত অনাথ যাকে কখনও কখনও ডেমনস বলে। সুতরাং, একটি ইচ্ছাকৃত অনাথ বা একটি ডেমন, ব্যবহারকারী সেশন থেকে আলাদা ভাবে এবং ব্যাকগ্রাউন্ডে রান করে। সুতরাং, এগুলি সাধারণত ব্যাকগ্রাউন্ড সেবা চালানোর জন্য ব্যবহৃত হয়।

(স্লাইডসময় পড়ুন: 25:42)

তাই এখন আমাদের একটি অপারেটিং সিস্টেমের দৃষ্টিকোণ থেকে exit সিস্টেম কলটি দেখবো। প্রথমে আমরা লক্ষ্য করি যে sbin স্ল্যাশ init যা সিস্টেমের প্রথম প্রসেস যেটি কখনও প্রস্থান করতে পারে না। অন্যান্য সব প্রসেসের জন্য, তারা exit সিস্টেম কল চালু করে অথবা একটি সংকেত দ্বারা অসম্মতভাবে বেরিয়ে আসতে পারে। সুতরাং, যখন এই প্রসেসটি প্রস্থান করে, অপারেটিং সিস্টেম নিম্নলিখিত অপারেশনগুলি করবে। প্রথমত, এটি সকল থোলা ফাইলের ব্যবহারের সংখ্যা হ্রাস করবে। এখন যদি ব্যবহার সংখ্যা 0 হতে হয়, তাহলে ফাইলটি বন্ধ হয়ে যায়। তারপর প্যারেন্ট জেগে উঠবে, যদি প্যারেন্টটি বর্তমানে sleeping স্টেট অবস্থায় থাকে, তাহলে প্যারেন্ট একটি runnable স্টেটে তৈরি হবে। সুতরাং, মনে রাখবেন যে runnable স্টেট কে রেডি স্টেট ও বলে। সুতরাং, কেন আমাদের প্যারেন্টকে জেগে উঠতে হবে? আমাদের প্রয়োজন প্যারেন্টকে জাগানো, কারণ প্যারেন্ট wait সিস্টেম কলের ফলে চাইল্ডের জন্য অপেক্ষা করছে। সুতরাং, আমাদের প্যারেন্টকে জাগাতে হবে, যাতে প্যারেন্ট চলতে পারে। তারপর সব শিশুদের জন্য যে বর্তমান exit প্রসেস আছে, অপারেটিং সিস্টেমের init প্রসেস সব চাইল্ডদের দত্তক গ্রহণ করবে। এবং শেষ পর্যন্ত, এটি exit প্রসেস Zombie স্টেটের মধ্যে সেট করবে। সুতরাং লক্ষ্য করুন যে প্রসেসটির কিছু দিক যেমন পেজ ডিরেক্টরি এবং কার্নেল স্ট্যাক প্রস্থানকালে ডি-এলোকেট হয় না। অপারেটিং সিস্টেমের মধ্যে এই মেটাডেটা প্যারেন্ট প্রসেস দ্বারা ডি-এলোকেট করা হয়, যার ফলে প্যারেন্ট ক্র্যাশ চাইল্ডদেরকে শুধরে দেয় যেন বিশেষ কোনও প্রসেস ক্র্যাশ করেছে বলে অনুমান করা হয়, পেজ ডিরেক্টরি এবং কার্নেল স্ট্যাক

অপারেটিং সিস্টেমের মধ্যে উপস্থিত থাকে। এইভাবে, প্যারেন্ট প্রসেসকে ক্র্যাশ করা চাইল্ড পেজের ডিরেক্টরি এবং কার্নেল স্ট্যাকের বিষয়বস্তু পড়তে দেয়, এইভাবে ডিবাগিং ঘটে।

(স্লাইডটাইপ পড়ুন: 27:48)

এখন, wait সিস্টেম কল এর অভ্যন্তরীণ সম্পর্কে দেখুন, তাই xv6 অপারেটিং সিস্টেমের সাথে সম্পর্কযুক্ত এই বিশেষ ফ্লোচার্টটি। যখন ইউজার স্পেসের মধ্যে একটি প্যারেন্ট প্রসেস wait সিস্টেম কল আহ্বান করে, এই নিম্নলিখিত সবুজ অপারেশনগুলো, অপারেটিং সিস্টেমের মধ্যে ঘটে। সুতরাং, প্রথমে একটি লুপ যা প্রত্যেকটি প্রসেসের p টেবিলে পুনরুক্তি করে এবং যাচাই করে যে প্রসেস p প্রসেসকরণ প্রসেসের একটি চাইল্ড কিনা। সুতরাং, এটি একটি চাইল্ড না হলে, তারপর আপনি p টেবিলের পরবর্তী প্রসেস গ্রহণ করবেন; যাইহোক, যদি এটি একটি চাইল্ড হয়, এটি একটি জম্বি হয় কিনা তা খুঁজে বের করার জন্য আমরা একটি অতিরিক্ত খোজ চালাব। সুতরাং, যদি চাইল্ডটি একটি জম্বি না হয় তাহলে প্যারেন্ট প্রসেস sleep স্টেটে থাকবে; যাইহোক, যদি চাইল্ডটি একটি জম্বি হয়, তাহলে আমরা কার্নেল স্ট্যাক ডি-এলকেট করব এবং পেজ ডিরেক্টরিকে মুক্ত করব, এবং wait সিস্টেম কল p এর pid ফিরিয়ে দেবে। সুতরাং, এই ভিডিওতে, আমরা প্রসেসটি সম্পর্কে দেখেছি, কিভাবে এটি তৈরি করা হয়েছে, এটি কিভাবে প্রস্থান করে এবং সিস্টেম কলগুলি যেমন wait, fork, exec, exit সিস্টেম কল। ধন্যবাদ।